TASK 13.10

1 Draw a diagram to show the contents of the queue after "F" has joined the non-empty queue shown in Figure 13.08.

2 Draw a diagram to show the contents of the queue after one item has left the non-empty queue shown in Figure 13.08.

This method involves a lot of moving of data. A more efficient way to make use of the slots is the concept of a 'circular' queue. Pointers show where the front and end of the queue are. Eventually the queue will 'wrap around' to the beginning. Figure 13.09 shows a circular queue after 11 items have joined and five items have left the queue.

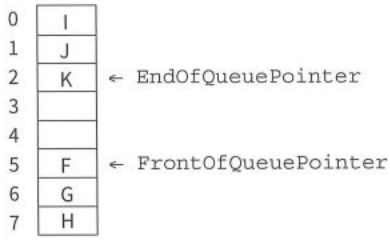


Figure 13.09 A circular queue

13.10 Linked lists

In Section 13.03 we used an array as a linear list. In a linear list, the list items are stored in consecutive locations. This is not always appropriate. Another method is to store an individual list item in whatever location is available and link the individual item into an ordered sequence using pointers.

An element of a list is called a node. A **node** can consist of several data items and a **pointer**, which is a variable that stores the address of the node it points to.

A pointer that does not point at anything is called a **null pointer.** It is usually represented by ∅. A variable that stores the address of the first element is called a **start pointer.**

KEY TERMS

Node: an element of a list

Pointer: a variable that stores the address of the node it points to

Null pointer: a pointer that does not point at anything

Start pointer: a variable that stores the address of the first element of a linked list

In Figure 13.10, the data value in the node box represents the key field of that node. There are likely to be many data items associated with each node. The arrows represent the pointers. It does not show at which address a node is stored, so the diagram does not give the value of the pointer, only where it conceptually links to.

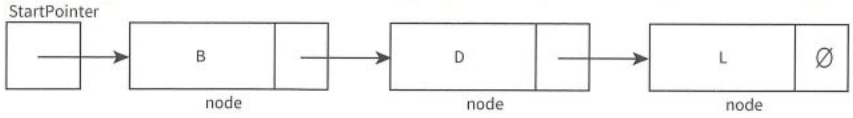


Figure 13.10 Conceptual diagram of a linked list

Figure 13.11 shows how a new node, A, is inserted at the beginning of the list. The content of `StartPointer` is copied into the new node's pointer field and `StartPointer` is set to point to the new node, A.

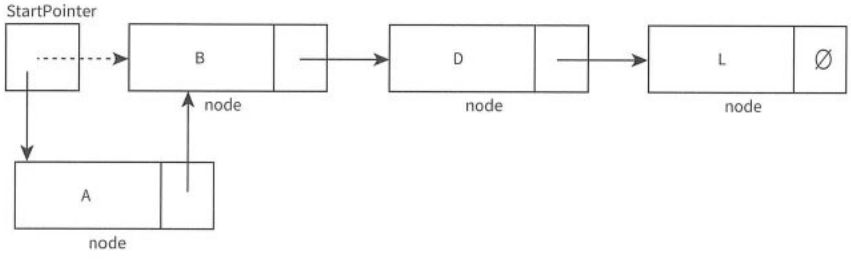


Figure 13.11 Conceptual diagram of adding a new node to the beginning of a linked list

In Figure 13.12, a new node, P, is inserted at the end of the list. The pointer field of node L points to the new node, P. The pointer field of the new node, P, contains the null pointer.

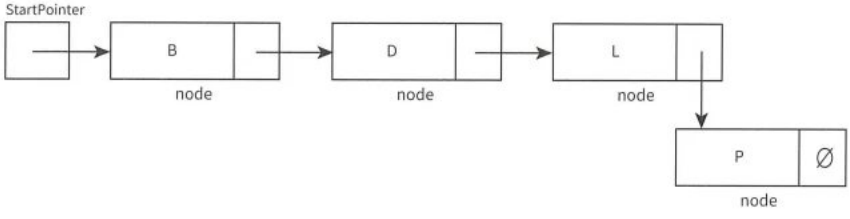


Figure 13.12 Conceptual diagram of adding a new node to the end of a linked list

To delete the first node in the list (see Figure 13.13), we copy the pointer field of the node to be deleted into `StartPointer`.

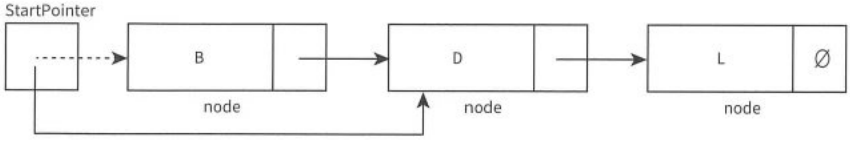


Figure 13.13 Deleting the first node in a linked list

To delete the last node in the list (see Figure 13.14), we set the pointer field for the previous node to the null pointer.

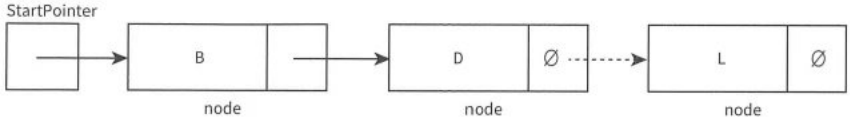


Figure 13.14 Conceptual diagram of deleting the last node of a linked list

Sometimes the nodes are linked together in order of key field value to produce an ordered linked list. This means a new node may need to be inserted or deleted from between two existing nodes.

To insert a new node, C, between existing nodes, B and D (see Figure 13.15), we copy the pointer field of node B into the pointer field of the new node, C. We change the pointer field of node B to point to the new node, C.

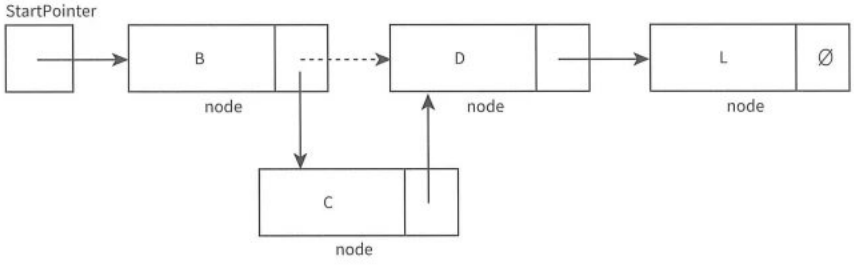


Figure 13.15 Conceptual diagram of adding a new node into a linked list

To delete a node, D, within the list (see Figure 13.16), we copy the pointer field of the node to be deleted, D, into the pointer field of node B.

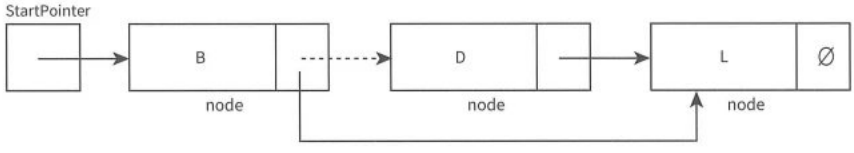


Figure 13.16 Conceptual diagram of deleting a node within a linked list

Remember that, in real applications, the data would consist of much more than a key field and one data item. This is why linked lists are preferable to linear lists. When list elements need reordering, only pointers need changing in a linked list. In a linear list, all data items would need to be moved.

Using linked lists saves time, however we need more storage space for the pointer fields.

To implement a linked list using arrays, we can use a 1D array to store the data and a 1D array to store the pointer. Reading the array values across at the same index, one row represents a node.

A value is added to the next free element of the `Data` array and pointers are adjusted to incorporate the node in the correct position within the linked list.

Figure 13.17 shows how two arrays can be used to implement the linked list from Figure 13.10.

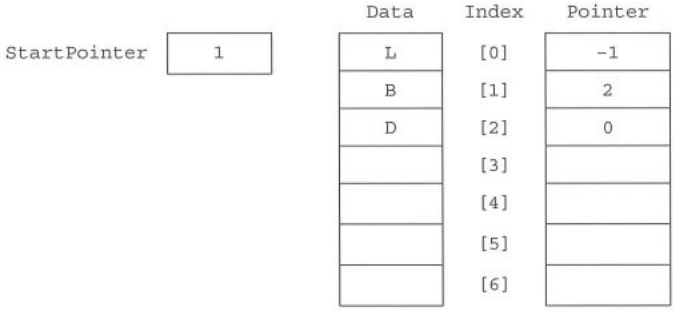


Figure 13.17 Using arrays to implement the linked list shown in Figure 13.10

Figure 13.18 shows how a new node is added to the beginning of a linked list implemented using arrays. Note that the value "A" is added at index 3 but the start pointer is adjusted to make this the new first element of the list.

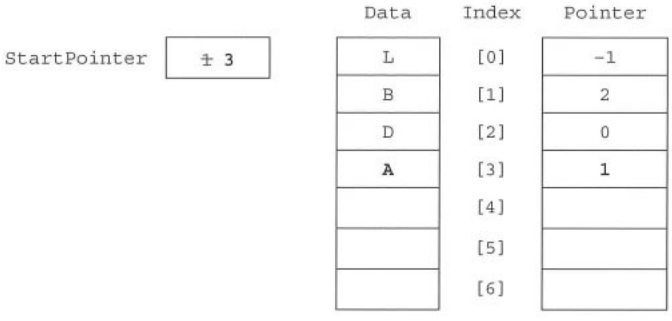


Figure 13.18 Adding a new node to the beginning of a linked list

Figure 13.19 shows how a new node is added to the end of a linked list implemented using arrays. Note that the value "P" is added at index 3. The node that previously contained the null pointer (at index 0) has its pointer adjusted to point to the new node.

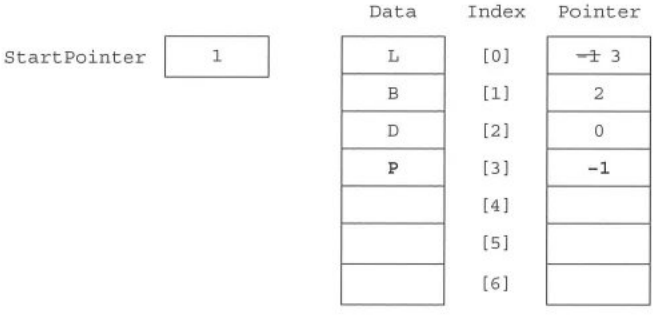


Figure 13.19 Adding a new node to the end of a linked list implemented using arrays

When deleting a node, only pointers need to be adjusted. The old data can remain in the array, but it will no longer be accessible as no pointer will point to it.

Figure 13.20 shows how the start pointer is adjusted to effectively delete the first element of the linked list. Note that the start pointer now contains the pointer value of the deleted node.

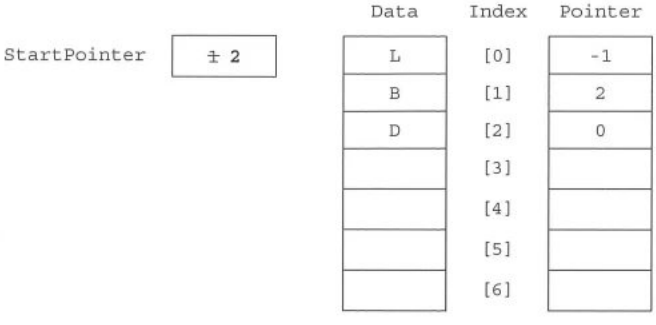


Figure 13.20 Deleting the first node in a linked list implemented using arrays

Figure 13.21 shows how the pointer value of the penultimate node of the linked list is changed to the null pointer.

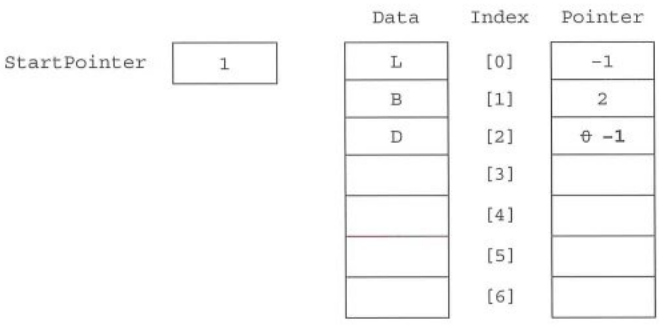


Figure 13.21 Deleting the last node of a linked list implemented using arrays

When adding a node that needs to be inserted into the list, the data is added to any free element of the Data array. The pointer of the new node is set to point to the index of the node that comes after the insertion point. Note that this is the value of the pointer of the node preceding the insertion point. The pointer of the node preceding the insertion point is set to point to the new node.

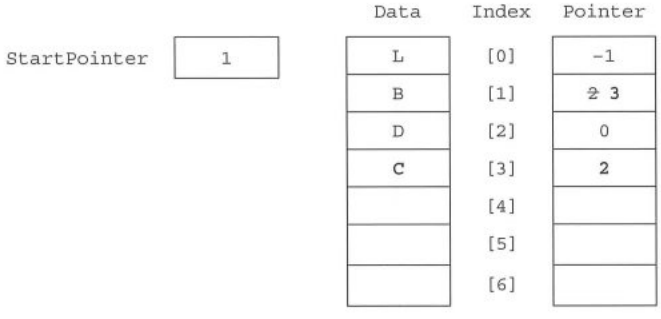


Figure 13.22 Adding a new node into a linked list implemented using arrays

Again, when deleting a node, only pointers need to be adjusted. Figure 13.23 shows how the pointer of the node to be deleted is copied into the pointer of the preceding node.

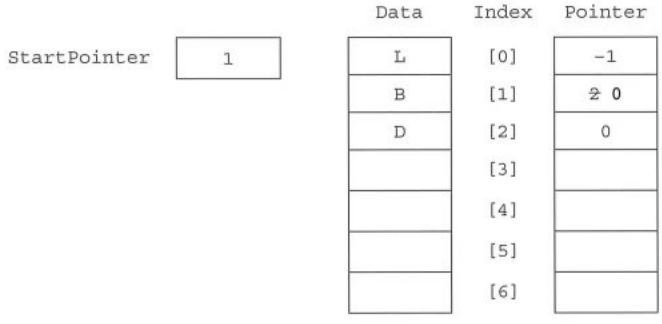


Figure 13.23 Deleting a node within a linked list implemented using arrays

Unused nodes need to be easy to find. A suitable technique is to link the unused nodes to form another linked list: the free list. Figure 13.24 shows our linked list and its free list.

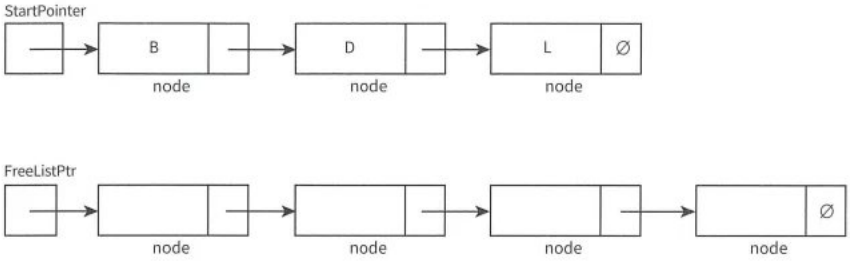


Figure 13.24 Conceptual diagram of a linked list and a free list

When an array of nodes is first initialised to work as a linked list, the linked list will be empty. So the start pointer will be the null pointer. All nodes need to be linked to form the free list. Figure 13.25 shows an example of an implementation of a linked list before any data is inserted into it.

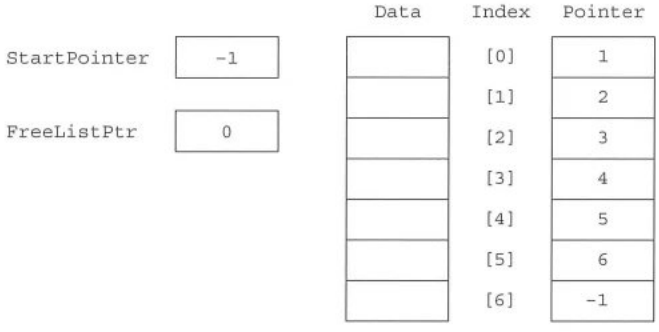


Figure 13.25 A linked list and a free list implemented using arrays

Assume "L", "B" and "D" were added to the linked list and to be kept in alphabetical order.

Figure 13.26 shows how the values are stored in the `Data` array and the pointers of the linked list and free list adjusted.

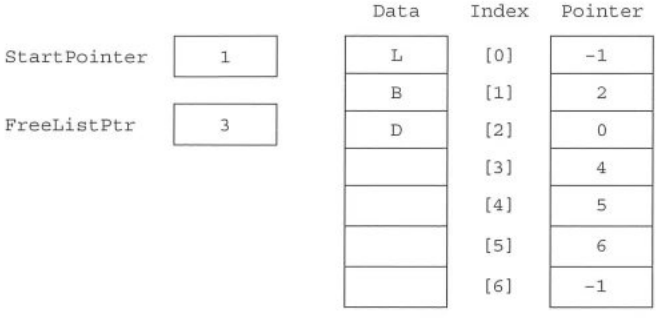


Figure 13.26 Linked list and free list implemented using arrays

If the node containing "B" is to be deleted, the array element of that node needs to be linked back into the free list. Figure 13.27 shows how this is done by adding the node to the front of the free list.

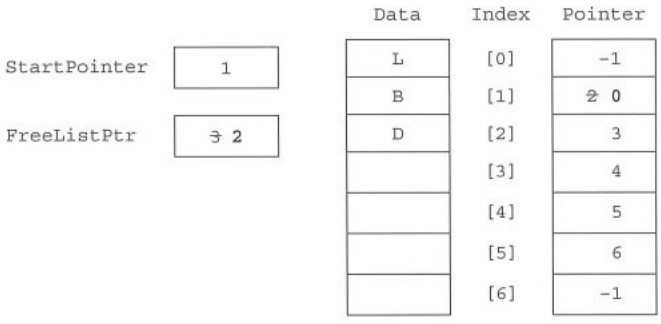
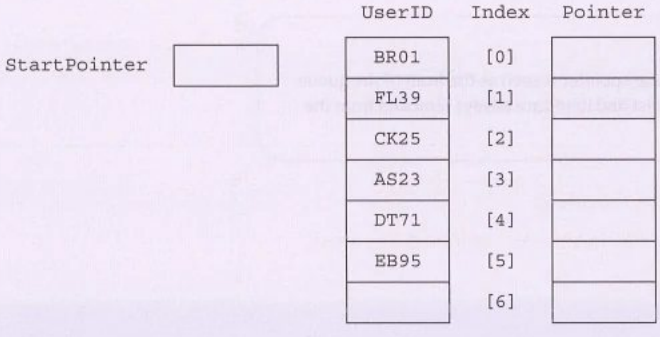


Figure 13.27 Linked list and free list implemented using arrays

TASK 13.11

A linked list is to be set up using the values in the `UserID` array shown below.



Without moving any of the contents of the `UserID` array, insert the pointer values so that the linked list is in alphabetical order.

In Section 13.02 we looked at the user-defined record type. We grouped together related data items into record data structures. To use a record variable, we first define a record type. Then we declare variables of that record type.

We can store the linked list in an array of records. One record represents a node and consists of the data and a pointer (see Figure 13.28).

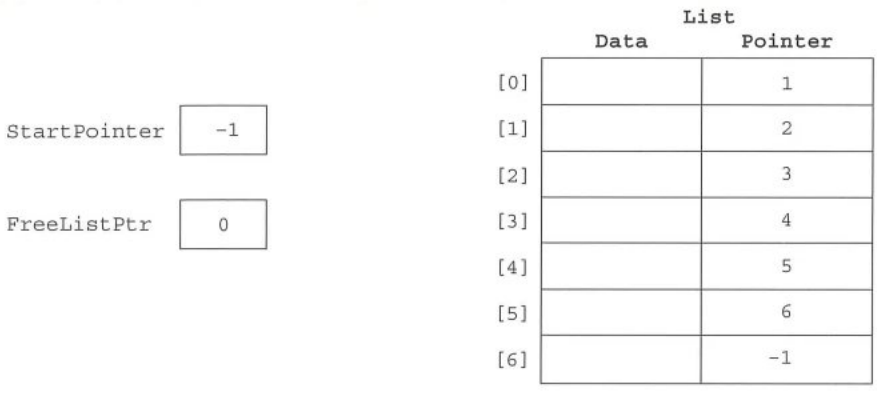


Figure 13.28 A linked list before any nodes are used

TIP

A stack can be implemented from a linked list. The start pointer is seen as the top of stack pointer and a data item is only added to the start of the linked list and a node is only removed from the start of the linked list.

TIP

A queue can be implemented from a linked list. The start pointer is seen as the front of the queue. Data items are always added to the end of the linked list and items are always removed from the start of the linked list.

Reflection Point:

What is the difference between standard data types, ADTs and user-defined data types?

Summary

- Standard data types are INTEGER, REAL, CHAR, STRING, BOOLEAN, DATE.
- A record structure holds a set of data of different data types under one identifier.
- Use the dot notation to address fields of a record.
- Arrays have dimensions with upper and lower bounds.
- Individual array elements are accessed using an index (1D arrays) or two indexes (2D arrays).
- A bubble sort algorithm compares pairs of values in a linear list and swaps them if required.
- A linear search checks each value in turn for a required value.
- Text files can be written to and read from and store data between program runs.
- Stacks, queues and linked lists are examples of Abstract Data Types (ADTs) and can be implemented using arrays.

Exam-style Questions

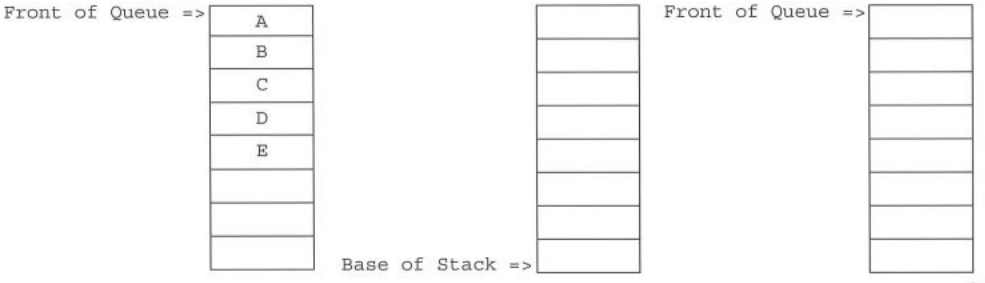
1 Complete the following variable identifier table:

Variable	Example value	Data type
`ColourCode`	***"034AB45"***	
`ProductionDate`	***2018/03/31***	
`Weight`	***67.45***	
`NumberInStock`	***98***	
`SizeCode`	***'X'***	
`Completed`	***FALSE***	

[3]

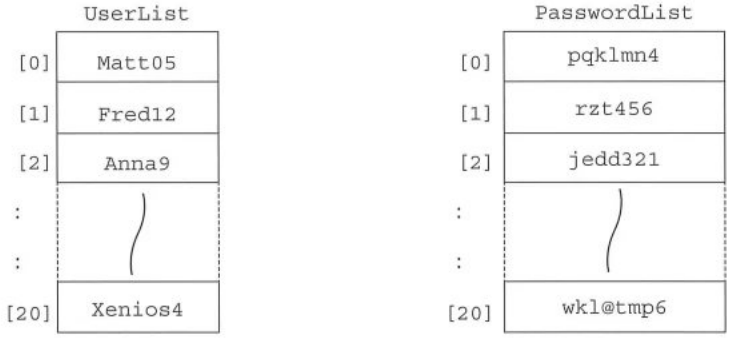
2 A stack and a queue are used to reverse the order of a set of values.

Complete the diagram:



[2]

3 Alicia uses two 1D arrays, `UserList` and `PasswordList`. For twenty users, she stores each user ID in `UserList` and the corresponding password in `PasswordList`. For example, the person with user ID `Fred12` has password `rzt456`.



Alicia wants to write an algorithm to check whether a user ID and password, entered by a user, are correct. She designs the algorithm to search `UserList` for the user ID. If the user ID is found, the password stored in `PasswordList` is to be compared to the entered password. If the passwords match, the login is successful. In all other cases, login is unsuccessful.

a Complete the identifier table. [8]

Identifier	Data type	Explanation
`UserList`		1D array to store user IDs
.....		1D array to store passwords
`MaxIndex`		Upper bound of the array
`MyUserID`		User ID entered to login
`MyPassword`		.....
`UserIdFound`		FALSE if user ID not found in UserList TRUE if .....
`LoginOK`		FALSE if ..... TRUE if .....
`Index`		Pointer to current array element

b Complete the pseudocode for Alicia's algorithm:

```
MaxIndex ← 20
INPUT MyUserID
INPUT MyPassword
UserIdFound ← FALSE
LoginOK ← ..............
Index ← -1
REPEAT
    INDEX ← ..............
    IF UserList[..............] = ..............
      THEN
        UserIdFound ← TRUE
    ENDIF
UNTIL .............. OR ..............
IF UserIdFound = TRUE
  THEN
    IF PasswordList[..............] = ..............
      THEN
        LoginOK ← TRUE
    ..............
ENDIF
IF ..............
  THEN
    OUTPUT "Login successful"
  ELSE
    OUTPUT "User ID and/or password incorrect"
ENDIF
```

[10]

c **i** Instead of using two 1D arrays, Alicia could have used an array of records.

Write pseudocode to declare the record structure `UserRecord`. [2]

ii Write pseudocode to declare the `User` array. [2]